Circuitos Aritméticos

Soma Binária

Sem *carry*

$$\begin{array}{r} 1 \\ +\quad 0 \\ \hline 1 \end{array}$$

Com *carry*

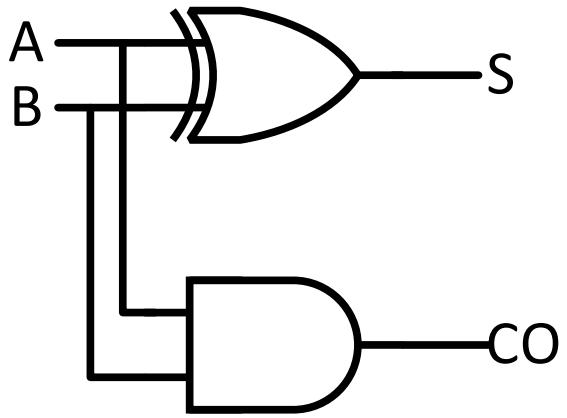
$$\begin{array}{r} 11 \\ +\quad 1 \\ \hline 1\,1\,0 \end{array}$$

Somador Binário – Semi-Somador (half-adder)

- Consideremos um circuito que soma dois bits, A e B.
- O resultado é um bit de soma, S, e um carry-out, CO
- A tabela de verdade do circuito é a seguinte:

A B	S	CO
0 0	0	0
0 1	1	0
1 0	1	0
1 1	0	1

$$S = A \oplus B$$
$$CO = A\,B$$



Soma Binária com Propagação do Carry

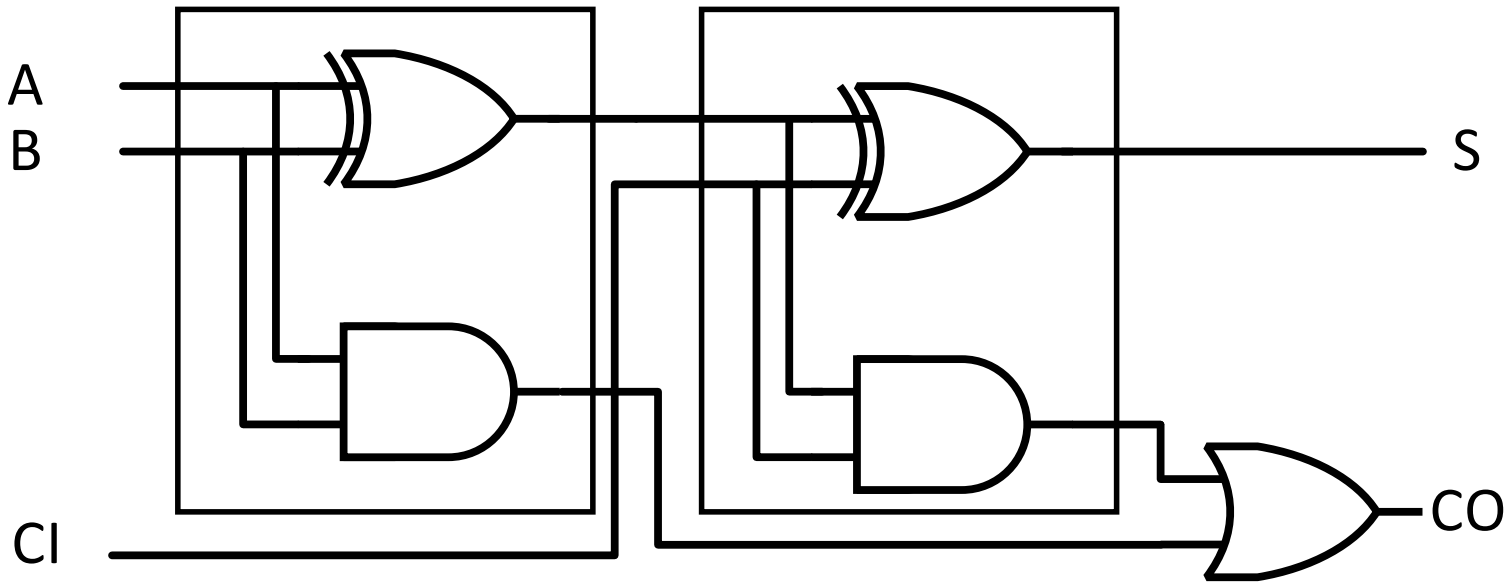
		1	1	0	0	- Cin
		1	0	1	1	
+			1	1	0	
	1	0	0	0	1	

Somador Binário–Somador Completo (full-adder)

- O *half-adder* não considera o *carry-in*, CI.
- A tabela de verdade do somador completo é a seguinte:

A B CI	S	CO
0 0 0	0	0
0 0 1	1	0
0 1 0	1	0
0 1 1	0	1
1 0 0	1	0
1 0 1	0	1
1 1 0	0	1
1 1 1	1	1

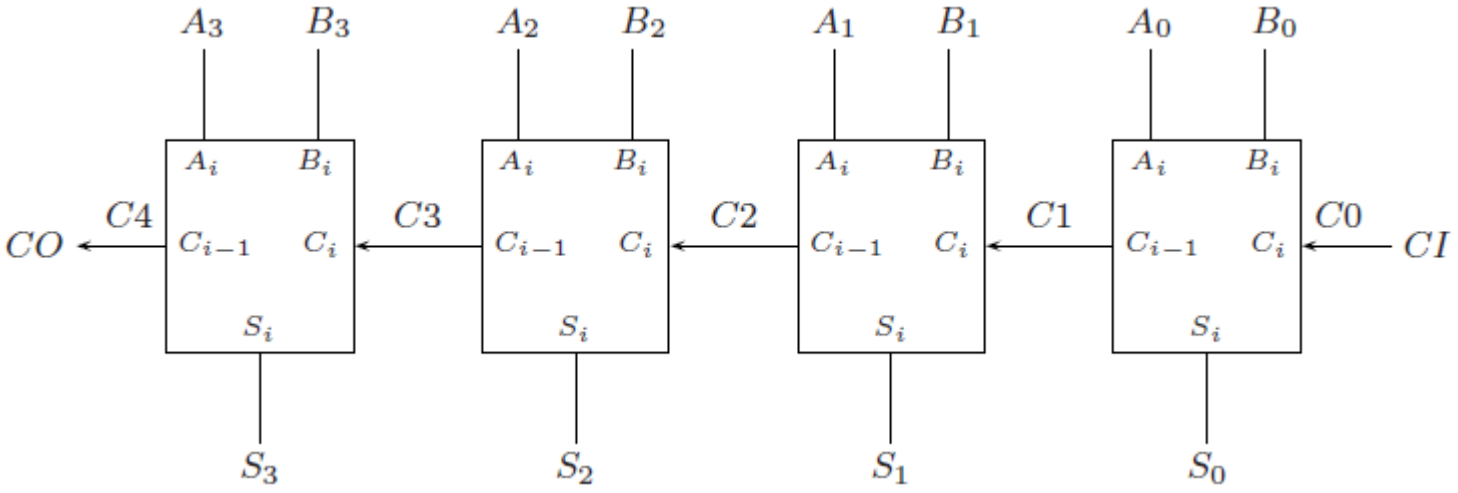
$$S = A \oplus B \oplus CI$$
$$CO = AB + CI(A \oplus B)$$

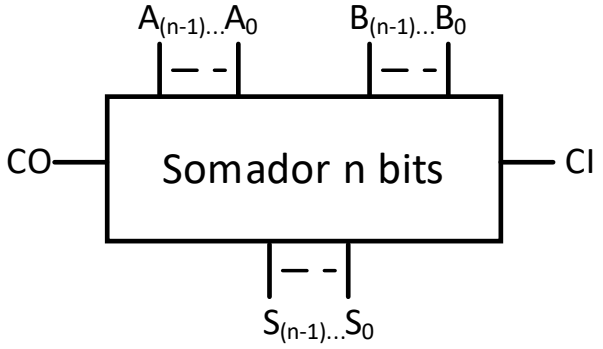


Somador Binário de *n* Bits

Se quisermos somar números de *n* bits usamos uma cadeia de somadores completos (*full-adders*)

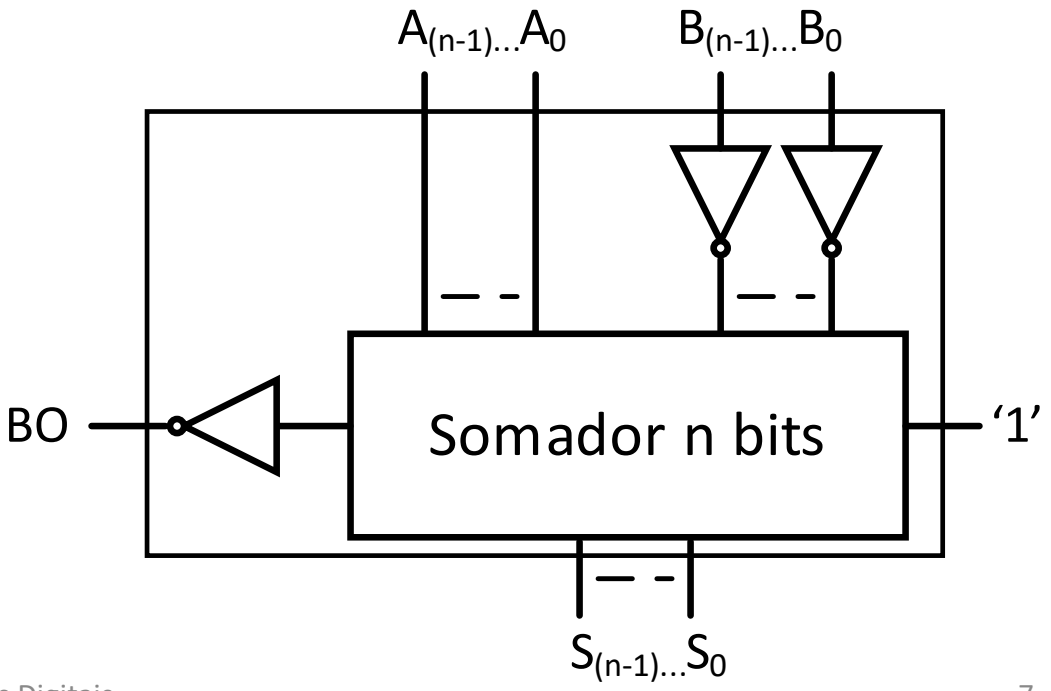
$$
\begin{array}{rcccc}
 & C_3 & C_2 & C_1 & CI \\
 & A_3 & A_2 & A_1 & A_0 \\
+ & B_3 & B_2 & B_1 & B_0 \\
\hline
CO & S_3 & S_2 & S_1 & S_0
\end{array}
$$





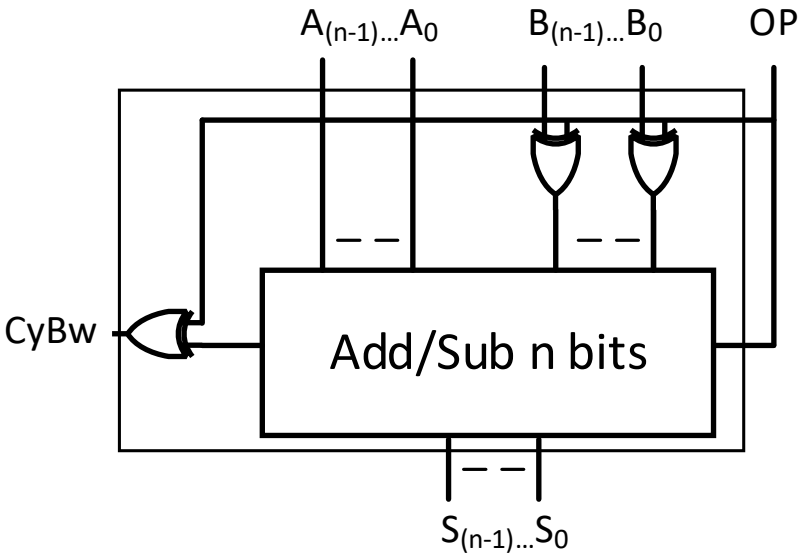
Subtrator de *n* Bits Com Somador

- Para implementar a subtração, A – B, com um somador, A + (-B) temos de somar A com o complemento para dois de B;
- Para calcular o complemento para dois de B:
  - Invertemos B – not B
  - Somamos 1 com o CI
  - Invertemos o CO (BO = not CO)



Somador/Subtrator de *n* Bits

- Um somador/subtractor calcula seletivamente a soma ou a subtração;
- Entrada, OP, define se executa a soma (OP=0) ou a subtração (OP=1);
- Para implementar um somador/subtrator, A + B ou A – B temos de:
- Complementar B em função de OP:
  - OP = 0, não complementa;
  - OP = 1, complementa;
- Controlar entrada CIN:
  - CIN = 0, se soma
  - CIN = 1, se subtração.
- Controlar saída de carry:
  - OP = 0, não complementa;
  - OP = 1, complementa.

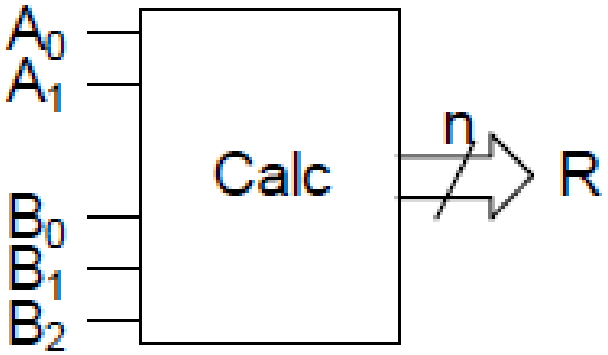


Geração das *Flags* – CY/BW, OV, GE

- A *flag* CY/BW indica se o resultado de uma operação com naturais está ou não dentro do domínio. A *flag* de carry retira-se da saída CO do somador. Numa subtração, BW = not CO;
- O overflow indica se o resultado de uma operação com relativos está ou não dentro do domínio. Pode ser gerado de duas formas:
  - $OV = C_{n-1} \oplus CO$
  - $OV = \overline{A_{n-1}}\ \overline{B_{n-1}}\ S_{n-1} + A_{n-1}\ B_{n-1}\ \overline{S_{n-1}}$
- O *grater or equal* (GE) indica se um úmero relativo é maior ou igual a outro. Pode ser determinado pelo sinal do resultado e pelo OV:
  - $GE = \overline{S_{n-1} \oplus OV}$

Circuito Aritmético 1 - Naturais

Utilizando somadores completos e lógica discreta, implemente o circuito da figura que calcula R = 3 x B + A, entendidos A e B como números naturais sem sinal. Cada somador deverá ter um número de bits estritamente necessário para garantir que os resultado, intermédios e final, nunca são excedidos.



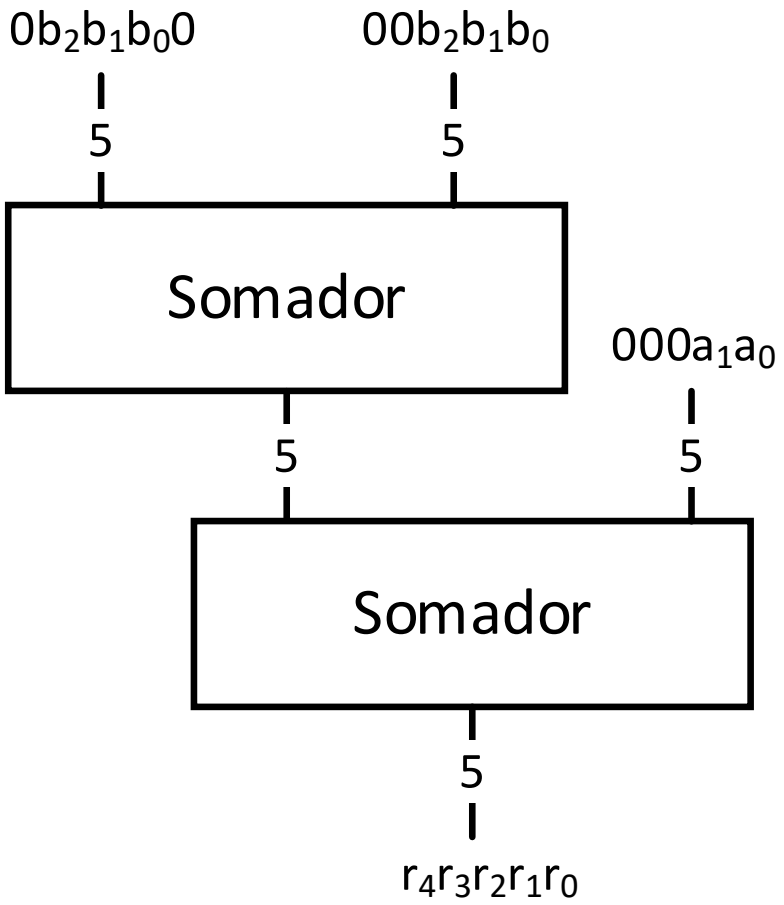
Circuito Aritmético 1 – Naturais
Resolução

R = 3 x B + A
A: [0, 3]
B: [0, 7]
R = [0, 24] (5 bits)
3 x B: [0, 21] (5 bits)

$B = b_2b_1b_0$ (3 bits)
$B = 00b_2b_1b_0$ (5 bits)

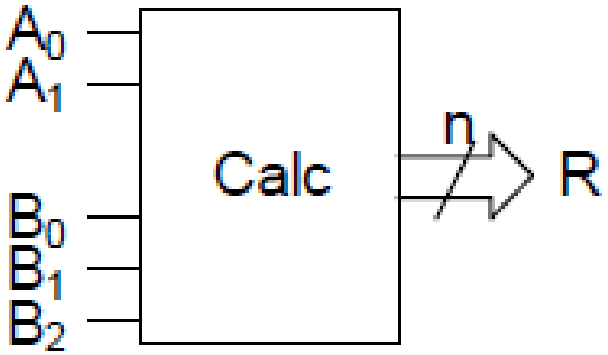
$2B = b_2b_1b_00$ (4 bits)
$2B = 0b_2b_1b_00$ (5 bits)

$A = a_1a_0$ (2 bits)
$A = 000a_1a_0$ (5 bits)



Circuito Aritmético 1 - Relativos

Utilizando somadores completos e lógica discreta, implemente o circuito da figura que calcula R = 3 x B + A, entendidos A e B como números inteiros com sinal representados em código de complementos. Cada somador deverá ter um número de bits estritamente necessário para garantir que os resultado, intermédios e final, nunca são excedidos.



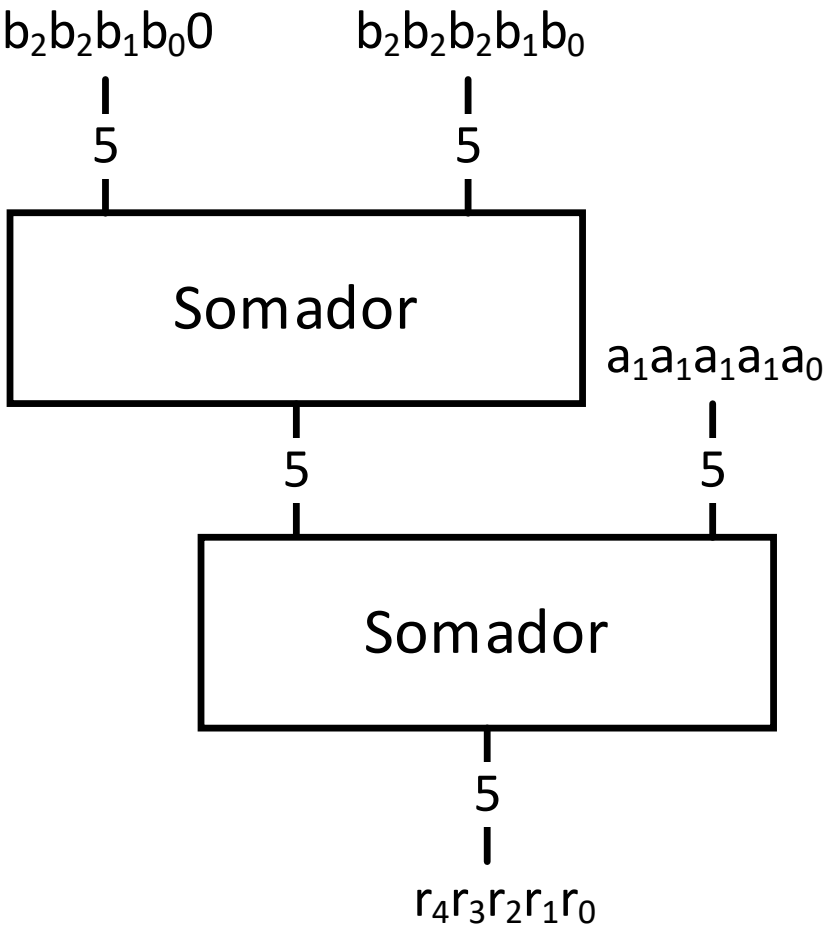
Circuito Aritmético 1 – Relativos Resolução

R = 3 x B + A
A: [-2, 1]
B: [-4, 3]
R = [-14, 10] (5 bits)
3 x B: [-12, 9] (5 bits)

B = $b_2b_1b_0$ (3 bits)
B = $b_2b_2b_2b_1b_0$ (5 bits)

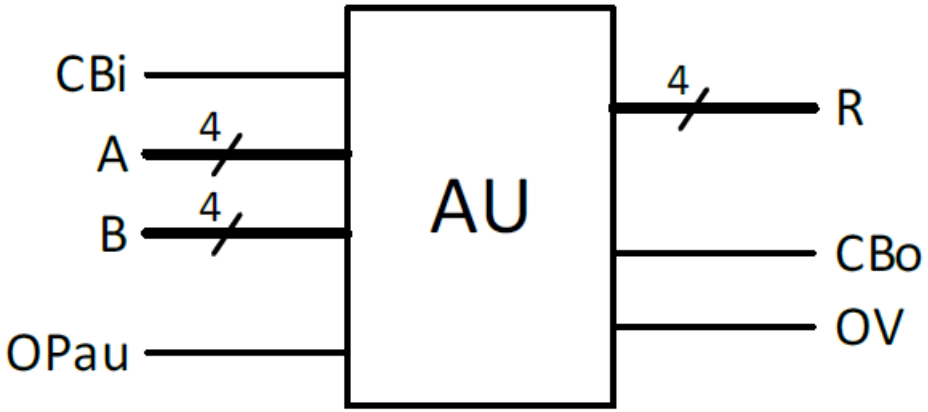
2B = $b_2b_1b_0$0 (4 bits)
2B = $b_2b_2b_1b_0$0 (5 bits)

A = a_1a_0 (2 bits)
A = $a_1a_1a_1a_1a_0$ (5 bits)



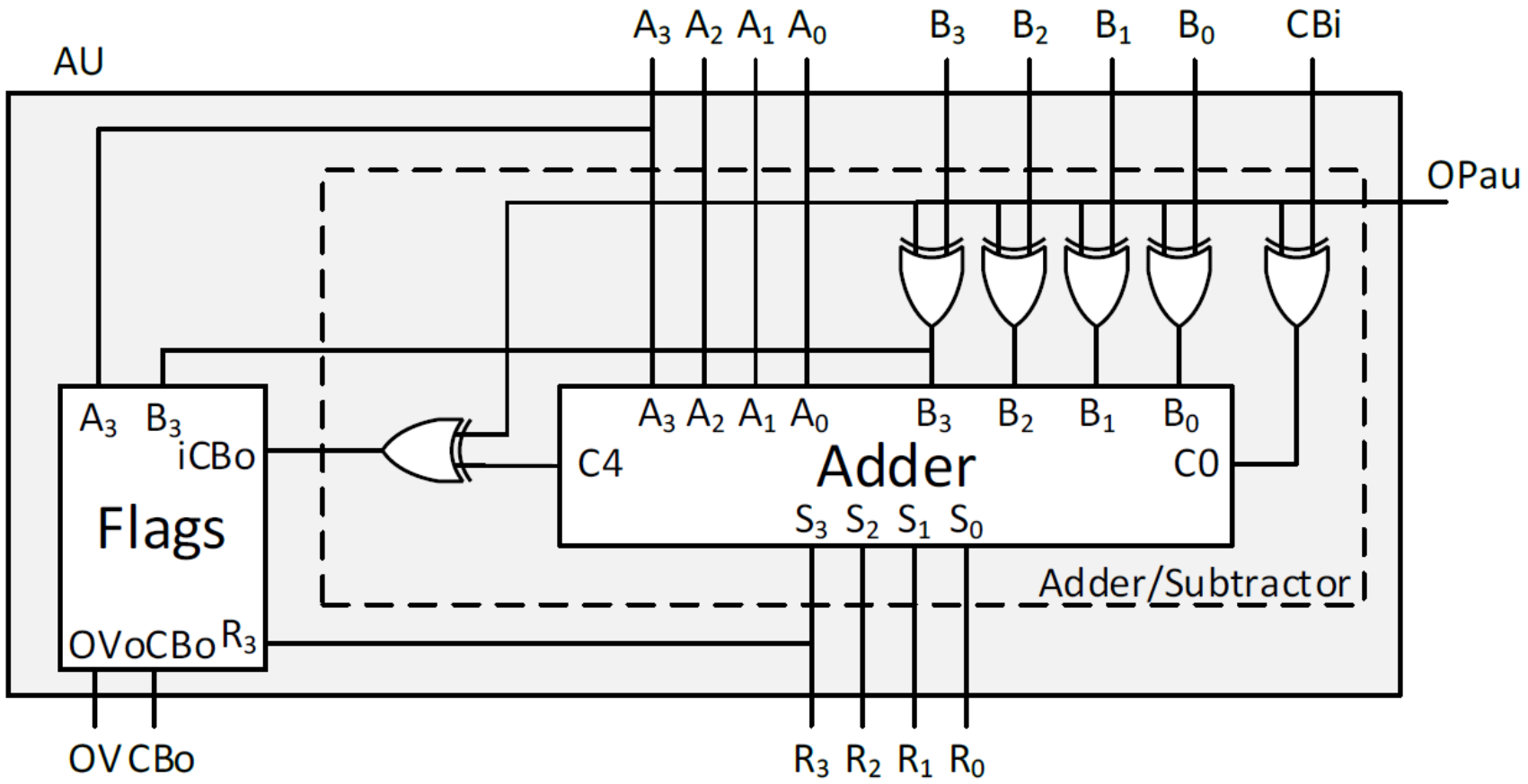
Circuito Aritmético - LABc

Projeto uma unidade aritmética que realize as operações +, - sobre operandos de 4 bits. O resultado tem 4 bits e deve gerar as flags Cbo e OV.



OPau	Operação
0	A + B + CBi
1	A – B – CBi

Circuito Aritmético 3 - LABc



Full-adder VHDL

```
LIBRARY IEEE;
use IEEE.std_logic_1164.all;

entity FULLADDER is
PORT (      A: in std_logic;
            B: in std_logic;
            Cin: in std_logic;
            R: out std_logic;
            Cout: out std_logic);
end FULLADDER;

architecture LOGICFULLADDER of FULLADDER is
begin
            R <= A xor B xor Cin;
            Cout <= (A and B) or ((A xor B) and Cin);
end LOGICFULLADDER;
```

Adder VHDL

```
LIBRARY IEEE;
use IEEE.std_logic_1164.all;

entity ADDER is
port (      A: in std_logic_vector(3 downto 0);
            B: in std_logic_vector(3 downto 0);
            C0 : in std_logic;
            S: out std_logic_vector(3 downto 0);
            C4: out std_logic);
end ADDER;
```

```
architecture ARCH_ADDER of ADDER is
component FULLADDER
PORT (      A: in std_logic;
            B: in std_logic;
            Cin: in std_logic;
            R: out std_logic;
            Cout: out std_logic);
end component;

signal C1,C2,C3: std_logic;

begin

FA0: FULLADDER port map (A => A(0), B=>B(0), Cin => C0, R=> S(0), Cout => C1);
FA1: FULLADDER port map (A => A(1), B=>B(1), Cin => C1, R=> S(1), Cout => C2);
FA2: FULLADDER port map (A => A(2), B=>B(2), Cin => C2, R=> S(2), Cout => C3);
FA3: FULLADDER port map (A => A(3), B=>B(3), Cin => C3, R=> S(3), Cout => C4);
end arch_adder;
```

Adder/Sub VHDL

```
LIBRARY IEEE;
use IEEE.std_logic_1164.all;

entity ADDSUB is
port (      A: in std_logic_vector(3 downto 0);
            B: in std_logic_vector(3 downto 0);
            Cbi : in std_logic;
            OP : in std_logic;
            R: out std_logic_vector(3 downto 0);
            B3 : out std_logic;
            CBo: out std_logic);
end ADDSUB;
```

```
architecture ARCH_ADDSUB of ADDSUB is
component ADDER is
port (      A: in std_logic_vector(3 downto 0);
            B: in std_logic_vector(3 downto 0);
            C0 : in std_logic;
            S: out std_logic_vector(3 downto 0);
            C4: out std_logic);
end component;

signal Cox,Cix: std_logic;
signal Bx : std_logic_vector(3 downto 0);

begin

A0: ADDER port map (A => A, B=>Bx, C0 => Cix, S=> R, C4 => Cox);

Cix <= Cbi xor OP;
Bx(0) <= B(0) xor OP;
Bx(1) <= B(1) xor OP;
Bx(2) <= B(2) xor OP;
Bx(3) <= B(3) xor OP;
Cbo <= Cox xor OP;
B3 <= Bx(3);

end arch_adder;
```

Flags VHDL

```
LIBRARY IEEE;
use IEEE.std_logic_1164.all;

entity FLAGS is
PORT (      A3: in std_logic;
            B3: in std_logic;
            R3: in std_logic;
            iCBo: in std_logic;
            OVo: out std_logic;
            CBo: out std_logic);
end FLAGS;

architecture ARCH_FLAGS of FLAGS is
begin
            OVo <= (not A3 and not B3 and R3) or (A3 and B3 and not R3);
            CBo <= iCBo;
end ARCH_FLAGS;
```

AU VHDL

```
LIBRARY IEEE;
use IEEE.std_logic_1164.all;

entity AU is
PORT (      A: in std_logic_vector(3 downto 0);
            B: in std_logic_vector(3 downto 0);
            CBi: in std_logic;
            OPau: in std_logic;
            R: out std_logic_vector(3 downto 0);
            CBo: out std_logic;
            OV: out std_logic);
end AU;

architecture ARCH_AU of AU is
Begin
(...)
end ARCH_AU;
```